



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Name: Devansh Singh
PRN: 25070122072
Batch: A-2
Division: A

Subject: Programming with Java

ASSIGNMENT No : 26

TITLE : Write a Java program to create and execute multiple threads using the Runnable interface.

Problem Statement

Write a Java program to create and execute multiple threads using the Runnable interface.

Learning Objectives

To integrate Java GUI applications with databases using JDBC.

Theory

The Runnable interface provides an alternative approach to creating threads in Java. A class implementing Runnable defines the run() method that contains the code executed by the thread. The Runnable object is passed to a Thread object, which starts execution using the start() method.

Unlike extending the Thread class, implementing Runnable allows a class to inherit from another superclass, making the design more flexible. Runnable separates task definition from thread execution, resulting in better code organization and reusability. It is widely used in multithreaded enterprise applications and thread pool implementations.

Output Screenshots

```
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp26/Main.java
Thread 2: 1
Thread 1: 1
Thread 2: 2
Thread 1: 2
Thread 2: 3
Thread 1: 3
Thread 2: 4
Thread 1: 4
Thread 2: 5
Thread 1: 5
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$
```

CODE

```
1 package Exp26;
2
3 class MyTask implements Runnable {
4     public void run() {
5         for (int i = 1; i <= 5; i++) {
6             System.out.println(Thread.currentThread().getName() + ": " + i);
7
8             try {
9                 Thread.sleep(500);
10            } catch (InterruptedException e) {
11                System.out.println(e);
12            }
13        }
14    }
15 }
16
17 public class Main {
18     public static void main(String[] args) {
19         MyTask task = new MyTask();
20
21         Thread t1 = new Thread(task, "Thread 1");
22         Thread t2 = new Thread(task, "Thread 2");
23
24         t1.start();
25         t2.start();
26     }
27 }
```

EXERCISE 1

```
1 package Exp26.Exercise;
2
3 class Task1 implements Runnable {
4     public void run() {
5         for (int i = 1; i <= 5; i++) {
6             System.out.println("Task 1: " + i);
7
8             try {
9                 Thread.sleep(500);
10            } catch (InterruptedException e) {
11                System.out.println(e);
12            }
13        }
14    }
15 }
16
17 class Task2 implements Runnable {
18     public void run() {
19         for (int i = 1; i <= 5; i++) {
20             System.out.println("Task 2: " + i);
21
22             try {
23                 Thread.sleep(500);
24            } catch (InterruptedException e) {
25                System.out.println(e);
26            }
27        }
28    }
29 }
30
31 public class Exp1 {
32     public static void main(String[] args) {
33         Thread t1 = new Thread(new Task1());
34         Thread t2 = new Thread(new Task2());
35
36         t1.start();
37         t2.start();
38     }
39 }
```

EXERCISE 2

```
1 package Exp26.Exercise;
2
3 class Bank {
4     int balance = 1000;
5
6     void deposit(int amount) {
7         balance += amount;
8         System.out.println("Deposited: " + amount);
9         System.out.println("Balance: " + balance);
10    }
11
12    void withdraw(int amount) {
13        balance -= amount;
14        System.out.println("Withdrawn: " + amount);
15        System.out.println("Balance: " + balance);
16    }
17 }
18
19 class Deposit implements Runnable {
20     Bank bank;
21
22     Deposit(Bank bank) {
23         this.bank = bank;
24     }
25
26     public void run() {
27         for (int i = 1; i <= 3; i++) {
28             bank.deposit(500);
29
30             try {
31                 Thread.sleep(500);
32             } catch (InterruptedException e) {
33                 System.out.println(e);
34             }
35         }
36     }
37 }
38
39 class Withdraw implements Runnable {
40     Bank bank;
41
42     Withdraw(Bank bank) {
43         this.bank = bank;
```

```

44     }
45
46     public void run() {
47         for (int i = 1; i <= 3; i++) {
48             bank.withdraw(200);
49
50             try {
51                 Thread.sleep(500);
52             } catch (InterruptedException e) {
53                 System.out.println(e);
54             }
55         }
56     }
57 }
58
59 public class Exp2 {
60     public static void main(String[] args) {
61         Bank bank = new Bank();
62
63         Thread deposit = new Thread(new Deposit(bank));
64         Thread withdraw = new Thread(new Withdraw(bank));
65
66         deposit.start();
67         withdraw.start();
68     }
69 }

```

Exercise

Q1. Create a program using `Runnable` to run two tasks simultaneously.

```

● dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp26/Exercise/Exp1.java
Task 1: 1
Task 2: 1
Task 1: 2
Task 2: 2
Task 2: 3
Task 1: 3
Task 2: 4
Task 1: 4
Task 2: 5
Task 1: 5
● dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$

```

Q2. Create a Java program using the `Runnable` interface where two tasks run simultaneously: one thread performs deposit operations, and another performs withdrawal operations, both displaying results concurrently.

```
● dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp26/Exercise/Exp2.java
Deposited: 500
Withdrawn: 200
Balance: 1300
Balance: 1300
Deposited: 500
Balance: 1800
Withdrawn: 200
Balance: 1800
Deposited: 500
Withdrawn: 200
Balance: 1600
Balance: 1600
❖ dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$
```

Conclusion

Thus, the experiment successfully demonstrated the creation and execution of multiple threads using the `Runnable` interface. The `run()` method was used to define the task, while the `Thread` class was used to start and execute the threads. The experiment also demonstrated simultaneous execution of multiple tasks and showed how `Runnable` provides a flexible approach to multithreading.